1주차 기술 검토 · TOPIC 2 / TOPIC 3

TDA4VM TIDL 모델 호환성 검토 보고서

Quantization(양자화) 지원 모델·Runtime(런타임) 조사 및 SK-TDA4VM 보드 호환성 Risk(위험 요소) 점검

과제	차량용 임베디드 플랫폼 기반 영상·IMU(Inertial Measurement Unit, 관성측정장치) 융합 딜레마존 통과 판정 시스템 개발
대상 보드	SK-TDA4VM (SoC(System on Chip): TDA4VM = J721E 계열)
기준 소프트웨어	Processor SDK(Software Development Kit) Linux 11.00.00.08 / edgeai-tidl-tools tag `11_00_06_00` / `SOC=am68pa`
검토 범위	Topic 2 — TIDL(TI Deep Learning)이 지원하는 Runtime·Framework(학습 프레임워크)·Quantization 지원 모델 조사 Topic 3 — 모델↔보드 호환성 Risk, 모델·보드 요구사항, Batch(배치) 적용 시 불일치 지점 점검
작성일	2026-10-05

0. 핵심 결론 요약
1. 검토 기준과 근거 자료
2. [Topic 2] TIDL Runtime·Framework·지원 모델
2.1 동작 구조 · 2.2 Runtime 종류
2.3 Framework 매칭과 Customizing 지점
2.4 저장소 예제 모델 · 2.5 TI Model Zoo
2.6 Quantization 지원 · 2.7 Operator 지원
3. [Topic 3] 보드 호환성 Risk 점검
3.1 보드 요구사항 · 3.2 모델 요구사항
3.3 Risk 목록 · 3.4 Batch 불일치 점검
3.5 과제 모델 적용 · 3.6 검증 절차 · 3.7 버전 선택
4. 결론 및 다음 작업
부록 A. Operator 목록 · B. Compile Option · C. 참고 문서

근거: TexasInstruments/edgeai-tidl-tools (tag 11_00_06_00 문서·예제 코드, 비교용 11_00_08_00 · 11_01_xx · 11_02_xx), TexasInstruments/edgeai-tensorlab (branch r11.0: edgeai-modelzoo, edgeai-benchmark), TI TDA4VM 제품 정보, 과제 수행계획서. 심각도(High/Medium/Low)는 문서 근거를 바탕으로 한 검토자 판단이다.

0. 핵심 결론 요약

edgeai-tidl-tools `11_00_06_00` tag의 문서·예제 코드를 기준으로, SK-TDA4VM(`am68pa`)에서 영상·IMU 융합 모델을 TIDL로 가속할 때 결정해야 할 사항과 미리 차단해야 할 Risk를 정리했다.

1. **Runtime 선택 — ONNX Runtime 경로 권장.** TIDL은 OSRT(Open Source Runtime) 3종(ONNX Runtime, TFLite Runtime, TVM/Neo-AI DLR(Deep Learning Runtime))과 그 하위 계층인 TIDL-RT(C API)를 제공한다. Operator(연산자) 매핑 표와 공식 예제가 갖춰진 경로는 **ONNX Runtime**과 **TFLite Runtime** 2가지다. 과제 모델은 PyTorch 학습·2-input 구조이고, TI Model Zoo 검출 모델과 Transformer 지원이 ONNX 중심이므로 ONNX Runtime + TIDL Execution Provider를 기본 경로로 둔다.
   → PyTorch → ONNX(opset ≤ 19, 고정 shape) → onnxruntime_tidl 1.15.0

2. **TDA4VM 전용 Quantization 제약.** TDA4VM(AM68PA)은 Symmetric(대칭) PTQ(Post-Training Quantization, 학습 후 양자화) 8/16-bit와 Mixed Precision(혼합 정밀도), ONNX QDQ(Quantize-Dequantize) 모델, QAT(Quantization Aware Training, 양자화 인지 학습)를 지원한다. 반면 **Asymmetric(비대칭)·Per-channel 양자화(`quantization_scale_type=4`)와 TFLite INT8 모델 직접 import(`=3`)는 지원하지 않는다.**
   → 다른 SoC(AM62A/AM68A/AM69A) 예제의 양자화 설정을 그대로 복사하지 않는다.

3. **IMU 분기의 LSTM/GRU는 가속 불가.** LSTM·GRU·RNN은 `11_00_06_00` 지원 Operator 목록에 없어 Cortex-A72(ARM)에서 실행된다. 이 Operator들은 `11_02_16_00` 부터 목록에 추가되었으나, 이를 포함한 tools(`11_02_16/17/20/21_00`) 중 SDK 11.00에 적용 가능한 패치는 J784S4 ADAS SDK 전용뿐이다. 즉 TDA4VM + SDK 11.00 조합에서는 사용할 수 없고, 최소 SDK 11.01 + `11_02_17_00` 이 필요하다.
   → IMU 시계열 분기는 Conv(1×k)·Gemm(완전연결) 기반으로 설계한다.

4. **Batch는 1로 고정해 배포.** TDA4VM은 C7x DSP(Digital Signal Processor)+MMA(Matrix Multiply Accelerator)가 1개이므로 multi-core Batch 모드(`inference_mode` 1/2)가 해당되지 않는다. 단일 DSP의 Batch 내부 최적화는 "매우 작은 해상도"에서만 적용되고, Slice는 batch=1만, Concat은 batch 축 불가, Add/Mul broadcasting은 batch 축 제외 등 Operator 제약이 있다.
   → 시간 축은 Batch가 아니라 W(width) 또는 C(channel) 축에 배치한다.

5. **검출(신호 인식) 후보는 TI Model Zoo의 YOLOX-lite.** edgeai-modelzoo r11.0의 YOLOX-nano/tiny/s-lite는 TIDL 후처리 가속(`meta_arch_type=6`)과 prototxt가 제공된다. 이름에 `gplv3` 가 붙은 YOLOv5/v6/v7/v8 변형은 shortlist에서 제외되어 있고, 라이선스 의무 검토가 필요하다.

6. **버전 고정이 최우선 Risk 관리 항목.** GitHub master(현재 `11_02_20_00`) 문서는 `11_00_06_00` 과 지원 기능이 다르다. Tools(컴파일 도구)와 보드 Firmware 버전이 다르면 Artifact(컴파일 산출물)가 실행되지 않는다.
   → 판단 근거는 tag 문서만 사용하고, compile 설정·버전을 실험 기록에 함께 남긴다.

1. 검토 기준과 근거 자료

1.1 기준 버전 조합

항목	값	근거
보드 SoC 환경 변수	`SOC=am68pa` (J721E(TDA4VM) → am68pa, `j721e` 입력 시 am68pa로 변환)	README.md “Supported Devices”, setup.sh
Tools ↔ SDK 호환	`11_00_06_00` ↔ Processor SDK Linux 11.00.00.08 / RTOS 11.00.00.06 (Default 호환)	docs/version_compatibility_table.md
C7x Firmware 기본값	`advanced_options:c7x_firmware_version = "11_00_00_00"`	examples/osrt_python/README.md
Compile 호스트	x86 PC, Ubuntu 22.04, Python 3.10 (Docker 권장). Compile은 보드에서 불가	README.md “Pre-requisites”, 지원 기능 표
OSRT 패키지 (x86)	onnxruntime_tidl 1.15.0, tflite_runtime 2.12.0, tvm 0.12.0, dlr 1.13.0	setup.sh (REL=11_00_06_00)
Python 의존성	onnx 1.14.0, numpy 1.23.0, protobuf 3.20.3, flatbuffers 1.12.0	requirements_pc.txt
검증된 모델 형식	ONNX 1.14.0, ONNX Runtime 1.15.0 (OPSET-19), TFLite (TensorFlow 2.12.0)	docs/supported_ops_rts_versions.md

1.2 참고한 자료와 사용 방식

자료	버전	사용 목적
edgeai-tidl-tools 문서·예제 코드	tag `11_00_06_00`	**판단 기준**. 지원 Operator, Quantization, Batch, 디버깅 절차
edgeai-tidl-tools 문서	tag `11_00_08_00`	같은 SDK용 패치 버전의 문서 정정 사항 확인
edgeai-tidl-tools 문서	`11_01_xx`, `11_02_xx` (master = `11_02_20_00`)	이후 버전에서 추가된 기능 식별(“현재 버전에서 불가” 판단용), 공통 오류 유형
edgeai-tensorlab (edgeai-modelzoo, edgeai-benchmark)	branch `r11.0`	TDA4VM 대상 사전 학습 모델, 검출 모델 구성, shortlist 정보
TI TDA4VM 제품 정보	—	하드웨어 연산 자원(코어 수·클럭·TOPS)
과제 수행계획서	—	과제 모델 구성(영상·IMU 융합, 신호 인식, 도달시간 예측), 기준 SW 버전

> **주의: GitHub 기본 브랜치 문서와 사용 버전 문서가 다르다**
>
> GitHub에서 저장소를 열면 master(`11_02_20_00`) 문서가 표시된다. 이 문서에는 LSTM/GRU/RNN, 비교 연산 (Where/Less/Greater 등), ReduceSum, Expand 등 `11_00_06_00` 에 없는 Operator가 포함되어 있고, 디렉터리 구조(`docs/operators.md` , `runtimes/` , `model-tools/`)도 다르다. 기능 판단은 반드시 `git checkout 11_00_06_00` 후의 문서로 한다.

2. [Topic 2] TIDL Runtime·Framework·Quantization 지원 모델

2.1 TIDL 동작 구조

TIDL은 학습이 끝난 모델 파일(`.onnx`, `.tflite`)을 x86 PC에서 Compile(분석·양자화·메모리 계획)하여 Artifact를 만들고, 보드에서는 OSRT가 이 Artifact를 읽어 지원되는 부분을 C7x DSP + MMA로 Offload(가속기 할당)한다. 지원되지 않는 Operator는 같은 OSRT가 Cortex-A72에서 실행한다(Heterogeneous Execution, 이종 실행). 지원 Operator가 연속된 구간 하나가 Subgraph(부분 그래프) 하나가 된다.

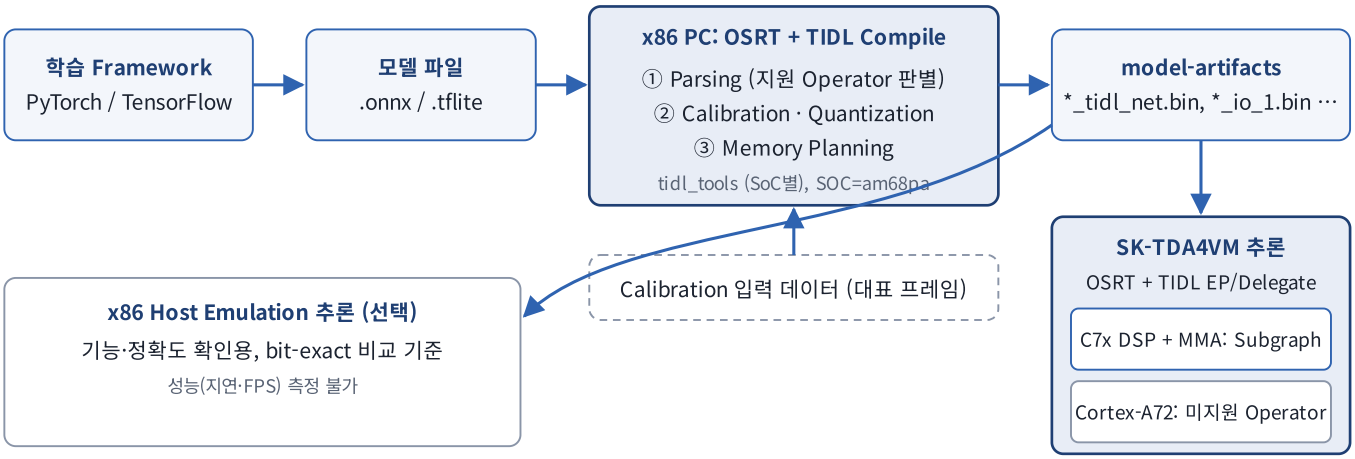


그림 1. TIDL Compile(x86) → Artifact → 추론(PC Host Emulation 또는 보드) 흐름 (README.md, examples/osrt_python/README.md 기반 재구성)

- Compile은 x86에서만 가능하다. 보드는 Inference(추론)만 수행한다(README.md 지원 기능 표).
- Host Emulation은 C7x 동작을 PC에서 재현하는 기능 확인 단계다. Layer별 실행 cycle은 보드에서 `debug_level=1`일 때만 출력된다(docs/tidl_osr_debug.md).
- Compile 결과로 `tempDir/runtimes_visualization.svg`가 생성되며, 어떤 노드가 C7x로, 어떤 노드가 ARM으로 배정되었는지 확인할 수 있다.

2.2 Runtime 종류와 사용 API

Runtime	입력 모델	Compile (x86)	가속 추론 (PC/보드)	11_00_06_00 비고
ONNX Runtime (ORT)	.onnx	providers = `['TIDLCompilationProvider', 'CPUExecutionProvider']`	providers = `['TIDLExecutionProvider', 'CPUExecutionProvider']`	onnxruntime_tidl 1.15.0 (OPSET-19). Vision Transformer는 ONNX만 지원. 예제: `examples/osrt_python/ort/onnxrt_ep.py`
TFLite Runtime	.tflite	`tflite.load_delegate('tidl_model_import_tflite.so', options)`	`tflite.load_delegate('libtidl_tfl_delegate.so', options)`	tflite_runtime 2.12.0, 입력 layout NHWC. 예제: `examples/osrt_python/tfl/tflrt_delegate.py`
TVM / Neo-AI DLR	.onnx / .tflite → Relay	`tidl.TIDLCompiler(platform="J7", …)` + `relay.build`	DLR	tvm 0.12.0 / dlr 1.13.0. `deny_list:layer_type` 미지원, `c7x_codegen`은 제한적 검증
TIDL-RT (C API)	OSRT Compile 결과	(별도 compile 없음, 위 OSRT로 생성)	`TIDLRT_create / invoke`	**전체 Offload된 모델만** 실행. `add_data_convert_ops=3`으로 compile 필요. ORT compile 시 출력 순서는 `onnxrtMetaData.txt`

> **"2가지 Runtime"의 실제 의미**
>
> 지원 Operator 매핑 표(docs/supported_ops_rts_versions.md)는 **ONNX**용과 **TFLite**용 두 가지만 있다. 즉 모델을 넣을 수 있는 실질적인 입구는 ONNX Runtime과 TFLite Runtime이며, 모델의 학습 Framework는 이 두 형식 중 하나로 내보낼 수 있어야 한다. TVM/DLR은 보조 경로이고, TIDL-RT는 두 Runtime이 내부적으로 호출하는 하위 계층(C API)이다.

2.3 Framework ↔ Runtime 매칭과 Code Customizing 지점

학습 Framework	변환 경로	대상 Runtime	주의 사항
PyTorch	`torch.onnx.export` (고정 입력 shape, opset ≤ 19) → onnxsim/Shape Inference → (필요 시) tidl-onnx-model-optimizer	ONNX Runtime	dynamic axes 미사용. ONNX Runtime 1.15.0이 OPSET-19까지 검증. Constant tensor가 있는 Add/Mul 등은 shape inference가 되어 있어야 함
TensorFlow / Keras	TFLiteConverter로 **float32** 모델 export	TFLite Runtime	TFLite full-integer(INT8) 모델 직접 import는 TDA4VM 미지원 → float 모델을 넘기고 TIDL PTQ 사용
edgeai-mmdetection (TI fork)	ONNX + prototxt(Object Detection Meta Architecture)	ONNX Runtime	TI Model Zoo 검출 모델의 표준 경로. 후처리 가속 사용 가능
Caffe	edgeai-caffe2onnx (requirements_pc.txt 포함) → ONNX	ONNX Runtime	예제 config(`cl-ort-caffe_*`)만 존재, 신규 개발에는 비권장
Ultralytics YOLO 원본	ONNX export 가능	ONNX Runtime	TIDL 친화 구조("lite")가 아님. 후처리·활성 함수 등 구조 변경 또는 TI 변형 사용 필요

Code Customizing이 필요한 경우

- **미지원 Operator 대체**: `osrt-model-tools/.../tidl_onnx_model_optimizer` 의 변환 규칙(예: `convert_resize_params_size_to_scale` , `convert_maxpool_to_cascaded_maxpool` , `convert_softmax_axis_*` , `convert_gemm_to_matmul_and_add` , `expand_layernorm_to_component_ops`) 또는 onnx-graphsurgeon으로 Graph(계산 그래프) 수정.
- **Offload 범위 조정**: `deny_list:layer_type` , `deny_list:layer_name` , `allow_list:layer_name` (실험적)로 특정 Operator를 ARM/C7x에 강제 배정.
- **다중 입력·비영상 입력 추론 코드**: 저장소 예제(`onnxrt_ep.py` , `tflrt_delegate.py`)는 분류·검출·분할의 단일 이미지 입력을 전제로 한다. 영상+IMU 2-input 모델은 docs/custom_model_evaluation.md의 "Custom Model Evaluation" 절차로 별도 추론 스크립트를 작성해야 한다.
- **전처리·후처리**: 입력 정규화(mean/scale)·layout 변환을 모델 안에 넣거나(`add_data_convert_ops` , 실험적) 애플리케이션에서 구현. 검출 후처리는 Meta Architecture(prototxt)로 TIDL에서 처리하거나 ARM에서 구현.

2.4 저장소 예제 모델 (11_00_06_00, `SOC=am68pa` 기준)

`examples/osrt_python/model_configs.py` 와 각 실행 스크립트의 기본 모델 목록에서 TDA4VM에 실제로 실행되는 항목만 정리했다.

Runtime	모델 키 (config 이름)	Task	비고
ONNX (기본)	`cl-ort-resnet18-v1`	Classification	ResNet18 (opset 9)
	`od-ort-ssd-lite_mobilenetv2_fpn`	Detection	meta_arch_type 3 (ONNX SSD)
	`ss-ort-deeplabv3lite_mobilenetv2`	Segmentation	am62a·am67a 제외 SoC에서 추가 → am68pa 포함
ONNX (`--run_model_zoo`)	`od-8020 … ssd_mobilenetv2_lite_512x512`	Detection	meta_arch_type 3
	`od-8200 … yolox_nano_lite_416x416`	Detection	meta_arch_type 6
	`od-8220 … yolox_s_lite_640x640`	Detection	meta_arch_type 6
	`ss-8610 … deeplabv3plus_mobilenetv2_edgeailite_512x512`	Segmentation	—
	`cl-6360 … regnetx-200mf`	Classification	—
TFLite (기본)	`cl-tfl-mobilenet_v1_1.0_224`	Classification	—
	`ss-tfl-deeplabv3_mnv2_ade20k_float`	Segmentation	float 모델
	`od-tfl-ssd_mobilenet_v2_300_float`	Detection	float 모델
	`od-tfl-ssdlite_mobiledet_dsp_320x320_coco`	Detection	meta_arch_type 1 (TFLite SSD)
AM69A 전용	`cl-ort-resnet18-v1_4batch` , `*_low_latency`	—	**TDA4VM 대상 아님** (`SOC == "am69a"` 일 때만 추가)

> **예제 기본 Compile 설정은 데모용 값이다**
>
> `examples/osrt_python/common_utils.py` 의 기본값은 `tensor_bits=8`, `accuracy_level=1`, `calibration_frames=2`, `calibration_iterations=5` 다. 문서 권장값(calibration_frames 최소 10, 기본 20 / advanced calibration 시 50)보다 훨씬 작으므로, 과제 모델에서 정확도를 평가할 때 그대로 쓰면 안 된다.

2.5 TI Model Zoo 후보 모델 (edgeai-tensorlab r11.0)

edgeai-modelzoo는 TI SoC에 맞게 일부 구조를 수정해 학습한 모델 모음이며, edgeai-benchmark에서 TDA4VM을 지원 SoC로 명시한다. 이름의 **lite**는 Depthwise/Grouped Convolution 사용, Swish·HSwish 대신 ReLU 사용, SE(Squeeze-and-Excitation) 모듈 제거를 의미한다(models/vision/detection/readme.md).

검출 모델	입력	AP[.5:.95] %	GigaMACS	meta_arch	비고 (edgeai-benchmark configs)
YOLOX-nano-lite	416	24.8	1.48	6	shortlisted, recommended. 저장소 예제 포함
YOLOX-tiny-lite	416	30.5	3.24	6	shortlisted
YOLOX-s-lite	640	38.3	13.43	6	shortlisted, recommended. 저장소 예제 포함
YOLOX-m-lite	640	44.4	36.9	6	shortlisted. 연산량이 큼
YOLOX-pico-lite	320	17.9	0.50	6	shortlisted, recommended. 정확도 낮음
MobileNetV2 + SSDLite	512	25.1	1.67	3	저장소 예제 포함
RegNetX200MF + SSDLite	320	20.7	0.61	3	저연산
EfficientDet-b0-lite	512	28.0	—	5	edgeai-mmdetection
YOLOv7-l-lite / YOLOv9-s-lite	640	45.9 / 38.3	—	6	YOLOv9-s는 shortlisted, YOLOv7-l은 not shortlisted
RTMDet-m/l-lite	640	—	—	—	shortlisted (2025 추가)
YOLOv5/v6/v7/v8 “-gplv3”	640	—	—	6 / 8	not shortlisted, edgeai-modelforest 경로. edgeai-yolov5·MMYOLO 저장소는 “유지보수 중단, 비권장”

- **영상 분기 Backbone(특징 추출부) 후보**: MobileNetV2/V3-lite, RegNetX-200/400/800MF, ResNet18/50 (classification 모델 목록, torchvision·edgeai-tv2 학습).
- **Transformer 계열**(ViT, DeiT, Swin, LeViT, DETR, Segformer)도 Model Zoo에 있으나, TIDL은 ONNX 모델만 지원하고 ONNX Runtime 최적화를 `ORT_DISABLE_ALL` 로 꺼야 하며, Swin 등은 8-bit 정확도 손실이 보고되어 있다(docs/tidl_fsg_vtfr.md).
- Model Zoo의 성능·정확도 보고서는 r10.1 tools와 firmware 10_01_04_00으로 생성되었다. `11_00_06_00` 에서의 수치는 직접 다시 측정해야 한다.

2.6 Object Detection Meta Architecture (후처리 가속)

구조	meta_arch_type	지원 export 경로
TFLite SSD	1	TensorFlow Object Detection API (TF1/TF2)
ONNX SSD	3	edgeai-mmdetection
YOLO V3	4	edgeai-mmdetection, open-mmlab mmdetection
RetinaNet / EfficientDet	5	edgeai-mmdetection, open-mmlab mmdetection
YOLO-X, YOLO-V5, YOLO-V7	6	edgeai-mmdetection, (MMYolo)
PointPillars 3D OD	7	edgeai-mmdetection3d
YOLO-V8	8	(MMYolo)

설정: `object_detection:meta_arch_type`, `object_detection:meta_layers_names_list` (prototxt 경로), 필요 시 `model_type="OD"`. 문서는 “지정된 export Framework로 만든 모델에서만 후처리 최적화가 지원된다”고 명시한다(docs/tidl_fsg_od_meta_arch.md).

2.7 Quantization 지원 방식 — TDA4VM(AM68PA) 기준

방식	설정	TDA4VM	근거·비고
PTQ, Symmetric, non-power-of-2 / power-of-2 scale	`advanced_options:quantization_scale_type` = 0 / 1	**지원**	모든 SoC 지원. 기본값 0
PTQ, Asymmetric · Per-channel	`quantization_scale_type = 4`	**미지원**	"all devices except TDA4VM/AM68PA" (tidl_fsg_quantization.md, Feature 비교 표)
TFLite full-integer 사전 양자화 모델	`quantization_scale_type = 3`	**미지원**	"available on all devices except for TDA4VM"
ONNX QDQ 사전 양자화 모델	`advanced_options:prequantized_model = 1`	**조건부**	지원 기능이나, TDA4VM은 비대칭 양자화를 지원하지 않으므로 대칭 QDQ로 만들어야 한다(검토자 판단, compile로 확인 필요)
QAT (Quantization Aware Training)	edgeai-modeloptimization(torchmodelopt)로 학습	**지원**	문서상 "TDA4VM Specific" 항목. 정확도 손실이 0에 가깝다고 기술
8-bit / 16-bit	`tensor_bits` = 8 / 16	**지원**	8-bit가 가장 빠름. 16-bit는 연산 시간 증가
Mixed Precision (수동)	`output_feature_16bit_names_list`, `params_16bit_names_list`	**지원**	정밀도 변경 가능 Layer: Convolution, BatchNorm, Pooling(Max 제외), EltWise
Mixed Precision (자동)	`mixed_precision_factor` (예: 1.2 = 8-bit 대비 1.2배 지연 허용)	**지원**	accuracy_level=1, calibration_frames/iterations=50 권장. Compile 시간 큼
32-bit float	`tensor_bits = 32`	**PC 전용**	Host Emulation 디버그 전용, 보드 실행 불가
LUT(Look-Up Table) 가속 비선형 활성	자동	**지원**	Sigmoid, HardSigmoid, GELU, TanH, Softmax, ELU (AM62A만 미지원)

Calibration(보정) 옵션 요약

- `accuracy_level`: 0 = 단순 min/max 보정, 1 = Advanced Bias Calibration(기본), 2 = calibration 프레임 전체 min/max, 9 = 사용자 지정(`activation_clipping`, `weight_clipping`, `bias_calibration`, `channel_wise_quantization`).
- Feature map(중간 출력) scale은 Simple Calibration에서 power-of-2만 지원한다. 50장 이상 사용 시 Bias Calibration 정확도 개선이 크다고 문서에 기술되어 있다.
- 회귀(regression, 연속값) 출력 모델(검출 box, 깊이 추정 등)은 첫·마지막 Convolution을 16-bit로 두는 것이 권장된다 — 과제의 도달시간·거리 예측 head에 해당한다.

2.8 Operator 지원 요약 (11_00_06_00)

ONNX 60종, TFLite 25종이 TIDL Layer로 매핑된다(전체 제약은 부록 A). 목록에 없는 Operator는 ARM에서 실행된다.

구분	Operator (ONNX)
C7x 가속 (주요)	Conv, ConvTranspose, Gemm, MatMul, BatchNormalization, Relu/PRelu/LeakyRelu/Clip/Sigmoid/Tanh/HardSigmoid/HardSwish/Mish/Elu, Add/Mul/Sub/Div/Max/Sum, MaxPool/AveragePool/GlobalAveragePool, Concat, Slice/Split, Reshape, Transpose, Flatten, Squeeze/Unsqueeze, Resize/Upsample, Softmax, ArgMax, Pad, DepthToSpace/SpaceToDepth, Gather, TopK, LayerNormalization, InstanceNormalization, GridSample, DeformConv, ScatterND/ScatterElements, ReduceMin/ReduceMax, Sqrt/Pow/Exp/Log/Abs/Floor/Neg, 삼각함수류, Cast(입출력단만), QuantizeLinear/DequantizeLinear(QDQ 모델)
단독 불가 (조합 시 가능)	ReduceMean(LayerNorm 조합 또는 optimizer로 MatMul 변환), Erf(GELU 조합), Dropout, Identity
11_00_06_00 미지원 (이후 버전에서 추가)	**LSTM, GRU, RNN**, GatherElements, NonZero (11_02_16_00~) · Where / Less / Greater / Equal / Not / And / Or / Xor / IsNaN / IsInf, Ceil, Round (11_02_13_00~) · ReduceSum, Expand (11_02_05_00~) · Min, Mod (최신 Eltwise 목록)
목록에 없음	Einsum, CumSum, Range 등 — ARM 실행

3. [Topic 3] 보드 호환성 Risk 점검

3.1 보드(TDA4VM / SK-TDA4VM) 요구사항

자원	사양	모델 배치(Deployment)상 의미
Cortex-A72 (ARM)	2코어, 최대 2.0 GHz	OSRT 실행, 미지원 Operator, 전처리·후처리, 판정 로직, 카메라·IMU 입력 처리가 모두 이 2코어를 공유
C7x DSP + MMA	C7x 최대 1.0 GHz, MMA 최대 8 TOPS(Tera Operations Per Second, 8-bit 기준)	TIDL Subgraph 실행. **1개**이므로 multi-core 추론 모드 없음. 8 TOPS는 8-bit 기준이며 16-bit는 처리량이 낮아짐
C66x DSP / R5F / GPU	C66x 2개(최대 1.35 GHz), Cortex-R5F 6개, PowerVR GPU	TIDL 대상 아님. 영상 전처리 등은 SDK의 별도 가속 경로 사용
메모리 (SK-TDA4VM)	LPDDR4 4 GB	모델·Artifact·카메라 버퍼 동시 적재. Memory Planning 실패 시 보드 실행 불가

하드웨어 사양 출처: TI TDA4VM 제품 정보(ti.com/product/TDA4VM). SK-TDA4VM 메모리 용량은 보드 수령 후 실물 확인 필요.

소프트웨어 측 보드 요구사항

요구사항	내용
Tools ↔ Firmware 일치	보드 SDK 11.00.00.08의 C7x Firmware와 같은 release line의 tools(`11_00_06_00`)로 compile. 다른 tools 사용 시 보드 Firmware·라이브러리 갱신(`update_target.sh`)과 `c7x_firmware_version` 지정 필요
SoC 일치	`tidl_tools`는 SoC별로 다르며 Artifact는 다른 SoC에서 사용 불가. `SOC=am68pa` 고정
OSRT 버전 일치	PC compile에 쓴 ORT·TFLite 버전과 보드 rootfs의 OSRT 버전이 같은 release line이어야 함 (backward compatible 패치 적용 시 보드의 wheel·라이브러리도 교체됨)
Graph 최적화 설정 일치	ONNX Runtime의 graph optimization level은 compile과 inference에서 같아야 함(최적화가 delegation 전에 graph를 바꾸기 때문)
디버그 로그	보드에서 C7x 오류 메시지를 보려면 `/opt/vision_apps/vision_apps_init.sh`로 원격 로그 출력 프로세스 실행 필요

SoC별 기능 차이 — TDA4VM이 다른 지점

기능	TDA4VM (AM68PA)	AM68A	AM69A	AM62A
Asymmetric · Per-channel Quantization	✗	✓	✓	✓
TFLite INT8 사전 양자화 모델 import	✗	✓	✓	✓
LUT 가속 비선형 활성	✓	✓	✓	✗
MatMul의 signed × unsigned 입력 조합	✗	—	—	—
Multi-DSP 추론 모드 (inference_mode 1/2)	✗ (DSP 1개)	✗	✓	✗
QAT 문서 항목	TDA4VM 전용 항목	—	—	—

"—"는 문서에 해당 SoC 제약 기술이 없음을 뜻한다. 출처: supported_ops_rts_versions.md(Feature 비교 표, MatMul 제약), tidl_fsg_quantization.md, tidl_fsg_multi_c7x.md (모두 11_00_06_00).

3.2 AI 모델 요구사항 (TIDL 11_00_06_00에 넣기 위한 조건)

ID	요구사항	세부 기준	확인 방법
M1	고정 입력 shape, batch = 1	dynamic axes 금지. Reshape는 "variable shape 미지원", Resize는 sizes 입력 대신 scale 사용	Netron, `onnx.shape_inference`
M2	ONNX opset ≤ 19	ONNX Runtime 1.15.0(OPSET-19), onnx 1.14.0 기준 검증. LayerNormalization-17 단일 op는 optimizer 문서상 미지원 사례 존재	`onnx.checker`, export 시 `opset_version` 명시
M3	Tensor 차원 수	Conv·Pool·Gemm·MatMul·Resize·ArgMax: non-singleton 차원 ≤ 4 / 그 외 대부분 ≤ 6 / Slice 가변 입력 ≤ 4	compile 로그(debug_level ≥ 1)

ID	요구사항	세부 기준	확인 방법
M4	지원 Operator·속성	부록 A 제약 충족. 위반 시 ARM 실행으로 Subgraph 분할 (최대 16개)	Parsing 요약, `runtimes_visualization.svg`
M5	Quantization 친화 학습	모든 Convolution(특히 Depthwise) 뒤 BatchNorm, weight decay ≈ 1e-4, ReLU 계열 활성. 마지막 예측 Conv는 BN 생략 가능	학습 config 점검
M6	회귀 출력 정밀도	도달시간·거리·속도 등 연속값 출력: 첫·마지막 Layer 16-bit 또는 `mixed_precision_factor`	8/16/32-bit 출력 비교
M7	전처리 일치	mean/scale을 학습과 동일하게. ONNX는 NCHW, TFLite는 NHWC	ARM-only 추론과 결과 비교
M8	Calibration 데이터	실제 입력 분포(주·야간, 정지·주행, 신호 상태)를 반영한 대표 프레임 20~50장 이상	목록·버전 기록
M9	검출 후처리 형식	meta_arch_type과 prototxt가 export Framework와 일치	compile 경고, 출력 tensor 확인
M10	전체 Offload 목표	단일 Subgraph가 성능상 유리하고, TIDL-RT C API는 전체 Offload 모델만 실행	Artifact의 net/io bin 개수

3.3 Risk 목록 (이 버전·이 보드에서 발생 가능한 오류)

심각도: **High** 과제 일정·구조에 직접 영향 · **Medium** 성능·정확도 저하 또는 수정 작업 필요 · **Low** 절차로 관리 가능.

ID	심각도	Risk	증상 / 오류 형태	대응
R01	High	Tools–Firmware 버전 불일치 (master 등 다른 tag로 compile)	보드 추론 실패, “Network version - X, Expected version - Y” 형식 오류 (11_02 debugging.md)	`11_00_06_00` 고정. 패치 버전 사용 시 `update_target.sh` + `c7x_firmware_version` 지정
R02	High	SOC 변수 오설정 (am68a 등)	Artifact가 보드와 비호환, 추론 실패	`SOC=am68pa` 를 스크립트·Docker 환경에 고정, Artifact 폴더명에 SoC 표기
R03	High	최신 문서 기준으로 기능 판단	11_00_06_00에 없는 Operator·옵션을 전제로 설계 → compile 단계에서 ARM 배정 발견	tag 문서만 근거로 사용 (본 보고서 2.8)
R04	High	IMU 분기에 LSTM/GRU/RNN 사용	해당 노드 ARM 실행, Subgraph 분할, C7x↔ARM 데이터 이동으로 지연 증가	Conv(1×k)·Gemm 기반 시계열 분기 (3.5). LSTM 필요 시 SDK 업그레이드 검토(3.7)
R05	High	TDA4VM 미지원 양자화 설정 사용	`quantization_scale_type=4` (비대칭) 또는 `=3` (TFLite INT8) 사용 → 미지원 기능이므로 compile 오류 또는 의도와 다른 양자화 결과	scale_type 0/1 사용, float 모델 + TIDL PTQ, 정확도 부족 시 QAT
R06	High	8-bit PTQ의 회귀 출력 정확도 저하	도달시간·거리 예측 오차 증가, 판정 경계 부근 오판	tensor_bits 32(PC) → 8 → 16 → Mixed Precision 순서로 비교, 출력단 16-bit
R07	Medium	미지원 Operator로 인한 ARM Fallback(대체 실행)	Subgraph 다수 생성(최대 16, 초과분 ARM), 지연 증가, Cortex-A72 부하 상승	optimizer 규칙, 구조 변경, `runtimes_visualization.svg` 로 매 compile 확인
R08	Medium	Convolution 형태 제약	stride는 H·W 동일해야 함 → stride (1,2) 시간축 Conv 불가 가능성. Depthwise kernel 제한, kernel > 7 & stride 2 불가, stride 4는 11×11만	stride 1 Conv + Pooling/GAP 조합, 일반 Conv 사용, 더미 모델로 사전 compile
R09	Medium	Axis(축) 제약 위반	Softmax는 W/H 축만, ArgMax는 axis=-3만, Concat은 batch 축 불가, ReduceMin/Max는 H 축만, LayerNorm은 W 축만, TopK는 H 축만, Pad는 W/H만	Fusion·head의 tensor 배치 사전 설계, optimizer의 softmax 축 변환 규칙
R10	Medium	MatMul 부호 조합 (TDA4VM)	signed × unsigned 입력 조합 및 signed 입력 → unsigned 출력 미지원 → attention·fusion MatMul에서 발생 가능	MatMul 두 입력의 활성 함수 통일, 가중치 상수인 Gemm 형태 사용

ID	심각도	Risk	증상 / 오류 형태	대응
R11	Medium	Resize·Upsample 제약	scale < 1, W/H scale 불일치, power-of-2 이외 scale, sizes 입력 → ARM 실행	2배 단위 upsample, `convert_resize_params_size_to_scale`
R12	Medium	Shape 관련 export 문제	dynamic axes, Shape→Gather→Reshape 연산 사슬 → “variable shape” 미지원	고정 shape export, onnxsim, shape inference
R13	Medium	Sub 연산의 두 가변 입력	11_00_06_00 문서는 미지원, 11_00_08_00 문서는 “firmware ≥ 10_01_06_00이면 지원”으로 정정 → 문서 간 불일치	IMU 차분 등 Sub 사용 시 더미 compile로 확인, 필요 시 Neg+Add로 대체
R14	Medium	부족한 Calibration	예제 기본값(frames=2, iterations=5) 사용 시 정확도 저하	대표 데이터 20~50장, accuracy_level=1, 데이터 목록 기록
R15	Medium	ORT graph optimization 설정 차이	compile·inference 간 graph 불일치, Transformer는 미설정 시 offload 실패	compile·추론 모두 같은 level, Transformer는 `ORT_DISABLE_ALL`
R16	Medium	검출 후처리 불일치	지원 export가 아닌 YOLO → NMS(Non-Maximum Suppression) 등 ARM 실행, 출력 형식 상이	edgeai-mmdetection 계열 + 제공 prototxt 사용
R17	Medium	Memory Planning 실패	8/16-bit compile 마지막 단계 오류, 보드 실행 불가 (32-bit는 이 단계 생략)	입력 해상도 축소, `high_resolution_optimization`, 모델 경량화
R18	Medium	다중 입력(영상+IMU) 처리	입력 순서 오류, 입력 type·scale 불일치, 예제 스크립트가 단일 이미지 입력만 처리	TIDL-RT는 `*_io_1.bin`의 `inDataName` 순서 사용, 별도 추론 코드 작성
R19	Medium	라이선스	“-gplv3” YOLO 변형, Ultralytics 원본 사용 시 공개 의무 → SW 등록 산출물과 충돌 가능	모델·학습 코드 라이선스 사전 확인 후 후보 확정
R20	Medium	ARM 자원 경합	Fallback Operator·IMU 전처리·후처리·판정 로직이 Cortex-A72 2코어 공유 → 전체 지연 증가	전체 Offload 목표, 보드에서 단계별 시간 측정
R21	Low	PC–보드 전처리 차이	float resize 등 결과 미세 차이로 layer trace bit-match 불가	raw binary 입력으로 비교, 전처리 고정
R22	Low	성능 수치 오해	Host Emulation 시간이나 Model Zoo(r10.1) 수치를 보드 성능으로 사용	보드에서 `debug_level=1` layer cycle 측정 후 판단

3.4 Batch 적용 시 불일치 점검

학습 시 Batch(N > 1)와 배포 시 Batch는 다르게 다뤄야 한다. TDA4VM에서 Batch를 사용할 때 맞지 않는 지점을 문서 근거와 함께 정리했다.

ID	점검 항목	내용	영향 / 대응
B1	Multi-core Batch 모드	`inference_mode=1` (High Throughput, 코어별 병렬 batch)· `=2` (Low Latency)는 DSP가 여러 개인 SoC(AM69A) 용. 예제의 4-batch 모델도 `SOC=="am69a"` 일 때만 실행	**TDA4VM 해당 없음**. `inference_mode=0`, `num_cores=1`
B2	단일 DSP의 Batch 최적화	Default 모드에서 batch > 1 모델은 내부 최적화를 하지만 “매우 작은 해상도에만 적용되고, 성능 이득이 없으면 적용되지 않음”	카메라 해상도 영상 분기는 batch 처리 시 지연이 batch 수에 비례해 증가 가능 → batch=1
B3	Slice / Split / StridedSlice	“Only batch size = 1 is supported”	batch > 1 모델에 Slice 존재 시 ARM 실행
B4	Concat	“Not supported along the batch dimension”, axis -3/-2/-1만	Batch 축으로 시퀀스·센서를 이어 붙이는 구조 불가 → channel 축 사용
B5	Add / Mul broadcasting	“Broadcasting on all dimensions except for the batch dimension”	[1,…] × [N,…] 형태 broadcasting 불가 → shape 사전 정렬
B6	Gather	“Input shape of dimension higher than axis should be 1”, indices 1D	batch 차원이 1이 아니면 조건 위반 가능

ID	점검 항목	내용	영향 / 대응
B7	BatchNormalization	`training_mode=1` 미지원	export는 `model.eval()` 상태에서 수행
B8	`advanced_options:batch_mode`	Batch stitching 옵션(기본 0). 최신 문서 설명: 일부 Layer에서 가장 바깥 non-singleton 차원을 width 차원으로 이어 붙여 성능 개선	11_00_06_00 문서에는 한 줄 설명만 존재 → 사용 시 출력 비교 필수
B9	optimizer `split_batch_dim_to_parallel_input_branches`	batch 차원을 병렬 branch로 분리. 문서상 “experimental, early stage”, 시작·끝 노드 이름 수동 지정 필요	기본 경로로 사용하지 않음
B10	Dynamic batch export	batch를 dynamic axis로 export하면 shape 미확정 → Reshape·Resize 제약 위반	배포용은 batch=1 고정 export (학습 batch와 별개)
B11	시간축 배치	IMU window(T 샘플)나 다중 프레임을 batch 축에 두면 B3~B6 제약에 걸림	IMU: [1, C, 1, T] (T를 W축), 다중 프레임: channel 축 stack
B12	처리량 확보 방식	TDA4VM에서는 batch 대신 프레임 단위 순차 추론 + 애플리케이션 단 pipeline이 적합	입력 수집·전처리(ARM)와 추론(C7x)을 겹쳐 실행

출처: tidl_fsg_multi_c7x.md, supported_ops_rts_versions.md, examples/osrt_python/README.md(batch_mode), osrt-model-tools/.../tidl_onnx_model_optimizer/README.md (11_00_06_00) · model_compilation.md, multi_c7x.md (11_02, batch_mode 설명).

3.5 과제 모델(영상·IMU 융합)에 적용한 설계 기준

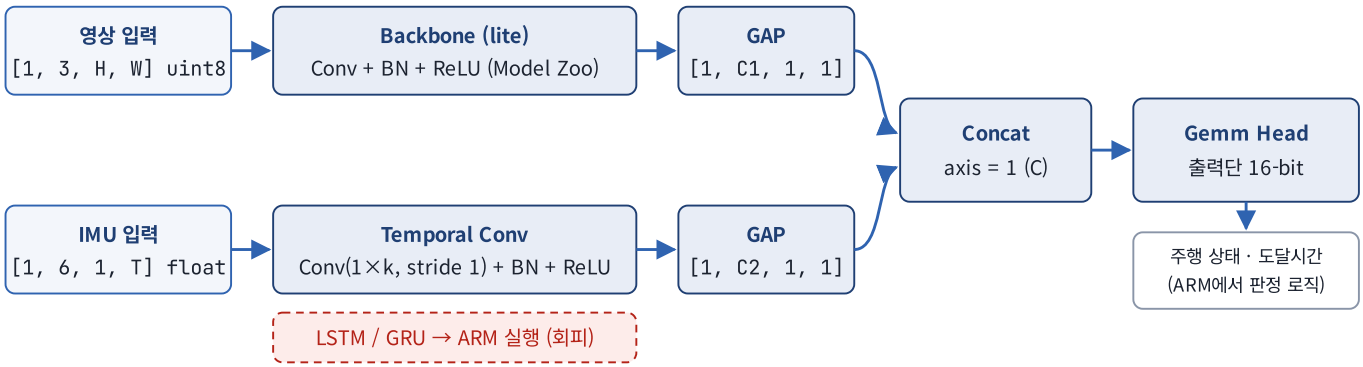


그림 2. 11_00_06_00 지원 Operator만으로 구성한 영상·IMU 융합 모델 예시 (검토자 제안, 더미 모델 compile로 검증 필요)

모듈	권장 구성	피할 구성	근거
영상 Backbone	Model Zoo lite backbone(MobileNetV2-lite, RegNetX, YOLOX-lite backbone), Conv 뒤 BN, ReLU	Swish/HSwish 다용, SE 모듈, 큰 kernel Depthwise	lite 정의, Depthwise kernel 제약
신호 인식 (검출)	YOLOX-nano/tiny/s-lite, 고정 입력 해상도, meta_arch_type 6 + prototxt	keep_ratio 가변 resize, 구조 변경 없는 Ultralytics 원본	od_meta_arch.md, detection readme
IMU 분기	[1, 6, 1, T] 배치 후 Conv(1×k, stride 1)+BN+ReLU 반복, 또는 Flatten → Gemm(MLP)	LSTM/GRU/RNN, stride (1,2), 가변 T	Operator 목록, Conv stride 제약
Fusion	각 분기 GAP → [1, C, 1, 1] → Concat(axis=1) → Gemm	batch 축 Concat·broadcast, signed×unsigned MatMul	Concat·Eltwise·MatMul 제약
출력 Head	Gemm 출력, Sigmoid/Tanh(LUT 가속) 사용 가능, 마지막 Layer 16-bit	ArgMax(axis≠-3), Softmax(C축)	Quantization 가이드, Softmax 제약
Runtime	ONNX Runtime + TIDL EP, 전체 Offload 목표, opt level 고정	Compile·추론 간 설정 불일치	tidl_osr_debug.md

> **검증이 필요한 가정**
>
> Conv(1×k) 비정사각 kernel은 일반 Convolution 제약 목록에 금지 조항이 없으나 명시적 검증 항목도 아니다. 또한 Pooling의 검증 kernel은 3×3·2×2·1×1이므로 1×2 같은 시간축 Pooling은 확인되지 않았다. GAP는 H×W ≤ 1024에서 그대로 쓰고, 초과 시 `convert_large_global_avg_pooling_to_matmul` 규칙을 적용한다. 본 구조는 실제 학습 전에 무작위 가중치의 더미 ONNX로 compile하여 전체 Offload 여부를 먼저 확인한다.

3.6 호환성 검증 절차

docs/tidl_osr_debug.md의 권장 순서를 과제 모델에 맞게 정리했다. 각 단계 결과는 실험 기록(모델 버전, compile option, calibration 데이터 목록, tools tag, SOC, ORT optimization level)과 함께 보관한다.

1. **ARM-only 기준 결과 확보** — TIDL 없이 `CPUExecutionProvider`(예제는 `-d` 옵션)로 추론해 기능상 정답 출력을 만든다.
2. **Compile 및 Offload 확인** — `debug_level=1~2` 로 compile 후 Parsing 요약, `tempDir/runtimes_visualization.svg`, Subgraph별 `*_tidl_net.bin.svg` 로 ARM 배정 노드와 원인(마우스 hover 정보)을 확인한다.
3. **32-bit 비교 (PC)** — `tensor_bits=32` 로 Host Emulation하여 1번 결과와 같으면 graph 변환은 정상이다(양자화 영향 배제).
4. **8-bit PTQ** — `accuracy_level=1`, calibration_frames 20~50으로 compile 후 정확도를 비교한다.
5. **16-bit · Mixed Precision** — 8-bit 정확도가 부족하면 16-bit로 상한을 확인하고, 수동(`output_feature_16bit_names_list`) 또는 자동(`mixed_precision_factor`)으로 조정한다. Layer별 오차는 `debug_level=4` trace로 비교한다.
6. **보드 성능 측정** — SK-TDA4VM에서 `debug_level=1` 로 layer cycle을 받아 병목 Layer를 찾는다(C7x 1 GHz 기준 cycle ÷ 10^6 = ms).
7. **PC–보드 일치 확인** — 양쪽에서 `debug_level=3` fixed-point trace(`/tmp`)를 생성해 비교한다. 첫 불일치 Layer가 보고 대상이다(`*.layer_info.txt` 로 dataId ↔ Layer 이름 대응).

3.7 버전 선택지 비교 (보드 SDK 11.00 유지 시)

Tools tag	호환 유형	필요 작업	변화
`11_00_06_00`	Default	없음	**현재 기준**
`11_00_08_00`	Patch (default 호환)	보드 Firmware·라이브러리 갱신, `c7x_firmware_version="11_00_08_00"`	Depthwise kernel 목록 정정(1x3s1, 5x5s1 포함), Sub 가변 입력 조건 정정 등
`11_01_05/07_00`, `11_02_05/07/13_00`	Patch (SDK 11.00 backward 호환)	`update_target.sh` 로 Firmware·라이브러리·OSRT 교체	ReduceSum·Expand(11_02_05~), 비교·논리 연산·Ceil·Round(11_02_13~) 추가. **11_02_13_00은 ONNX Runtime 1.23.0(OPSET-21)**으로 변경되어 PC·보드 환경 동시 변경. LSTM/GRU 없음
`11_02_16_00` 이상	SDK 11.02.01 이상 Default (`11_02_17_00` 은 SDK 11.01·11.02.00에 backward 호환)	보드 SDK 자체 변경 (최소 11.01)	LSTM/GRU/RNN 포함. SDK 11.00 대상 패치(`11_02_17_00`, `11_02_21_00`)는 J784S4 ADAS SDK 한정

> **권장**
>
> 1주차~기준 모델 검증(10/8~10/15)까지는 `11_00_06_00` 을 유지한다. 필요한 Operator가 이 버전에 없는 경우에만 위 표의 버전 변경을 팀 결정 사항으로 올린다. 버전 변경은 PC Docker 환경과 보드 Firmware를 함께 바꾸므로, 변경 시점 이전에 생성한 Artifact는 모두 재생성해야 한다.

4. 결론 및 다음 작업

4.1 결론

- **Runtime·Framework**: PyTorch 학습 → ONNX(opset ≤ 19, 고정 shape, batch=1) → ONNX Runtime + TIDL EP를 기본 경로로 한다. TensorFlow를 쓰는 경우 float TFLite → TFLite Runtime 경로를 쓰되 INT8 TFLite import는 TDA4VM에서 쓸 수 없다.
- **Quantization**: Symmetric PTQ 8-bit를 기본으로 하고, 회귀 출력은 16-bit·Mixed Precision으로 보완한다. 정확도 목표 미달 시 TDA4VM 대상 QAT를 검토한다.
- **모델 구조**: 영상 분기는 TI Model Zoo lite 구조, 신호 인식은 YOLOX-lite 계열, IMU 분기는 LSTM/GRU 대신 Conv(1×k)·Gemm, Fusion은 channel 축 Concat으로 구성하면 11_00_06_00 지원 Operator 안에서 전체 Offload를 목표로 할 수 있다.
- **Batch**: TDA4VM에서는 batch=1 배포가 기준이며, 시간·센서 정보는 batch가 아닌 W/C 축으로 표현한다.
- **버전**: Tools·Firmware·SoC·OSRT 버전을 한 묶음으로 기록·고정하는 것이 가장 큰 오류 원인을 제거한다.

4.2 다음 작업 (제안)

No	작업	완료 기준	관련 Risk
1	11_00_06_00에서 ONNX 기본 예제 3종 + `--run_model_zoo`의 YOLOX-nano/s-lite compile·PC 추론	Artifact·출력 이미지 생성, 설정 기록	R01–R03
2	IMU 분기 후보(Temporal Conv, MLP, 비교용 GRU) 더미 ONNX 생성 → compile	Offload 비율·Subgraph 수 비교표	R04, R08, R13
3	영상+IMU 2-input Fusion 더미 모델 compile, 커스텀 추론 스크립트 작성	단일 Subgraph 여부, 입력 순서 확인	R09, R10, R18
4	tensor_bits 32/8/16 비교 실험 템플릿 작성 (회귀 출력 오차 지표 포함)	비교 스크립트·결과 표 양식	R05, R06, R14
5	Calibration 데이터 선정 기준 정의 (주·야간, 신호 상태, 주행 상태)	데이터 목록 문서	R14
6	후보 모델·학습 코드 라이선스 확인	후보별 라이선스 표	R19
7	보드 수령 후 SDK·Firmware 버전 확인, `debug_level=1` layer cycle 측정	보드 성능 기준표	R01, R20, R22

부록 A. 11_00_06_00 Operator 지원 목록 (ONNX)

출처: docs/supported_ops_rts_versions.md (tag 11_00_06_00). “dim ≤ n”은 non-singleton 가변 입력 차원 수 제한. 표기된 제약 외 조건은 원문 확인.

No	ONNX Operator	TIDL Layer	핵심 제약
1	Conv	Convolution	가변 입력 1개, dim ≤ 4, stride H=W, kernel > 7 & stride 2 불가, Depthwise는 3x3s1·3x3s2·5x5·7x7s1·7x7s2 (11_00_08 문서: 1x3s1·3x3s1·3x3s2·5x5s1·5x5s2·7x7s1·7x7s2), stride 4는 11x11만, 입력 폭 < max(pad L, pad R) 불가
2	AveragePool / GlobalAveragePool / MaxPool	Pooling	dim ≤ 4, dilation 1, storage_order 0, 검증 kernel 3x3·2x2·1x1(stride 1/2), GAP H×W > 1024 → optimizer 규칙
3	Relu	ReLU	dim ≤ 6
4	PRelu	PReLU	dim ≤ 6, 가변 slope 불가
5	Sum / Add / Mul / Div / Sub / Max	EltWise	입력 2개, dim ≤ 6, broadcast 가능(batch 축 제외), Sub 두 입력 모두 가변 불가(11_00_08 문서 정정 참조), 상수 tensor는 shape inference 필요, 1D 벡터는 channel 또는 width와 일치
6	Gemm	InnerProduct	dim ≤ 4, transA=0·alpha=1·beta=1, bias는 [N] 또는 [1,N] (아니면 MatMul+Add 변환)
7	MatMul	InnerProduct	dim ≤ 4, signed 입력→unsigned 출력 불가, **signed×unsigned 입력 조합 TDA4VM 불가**
8	Softmax	SoftMax	dim ≤ 6, width·height 축만
9	BatchNormalization	BatchNorm	training_mode=1 불가, dim ≤ 6
10	ConvTranspose	Deconv2D	4x4·2x2 kernel & stride 2만, group=1, Resize 대체 권장
11	Concat	Concat	axis -3/-2/-1만, batch 축 불가, dim ≤ 6
12	Slice / Split	Slice	입력 4개(가변 1 + 상수 1D 3), 가변 dim ≤ 4, batch=1만, stride ≠ 1 단독 불가(Patch Merging 제외)
13	Flatten	Flatten	dim ≤ 6
14	Dropout	DropOut	단독 불가
15	ArgMax	ArgMax	keepdims=1, axis=-3만, dim ≤ 4, select_last_index=0
16	Upsample / Resize	Resize	dim ≤ 4, nearest·linear, W/H 축만, scale ≥ 1, W=H scale, power-of-2 scale, antialias=0, coordinate mode는 half_pixel·pytorch_half_pixel·asymmetric 일부 조합만
17	DepthToSpace	DepthToSpace	4D, channel이 block² 배수
18	Sigmoid	Sigmoid	dim ≤ 6
19	Pad	Pad	최대 6차원, constant mode만, W/H 축만
20	ReduceMin / ReduceMax	Reduce	height 축만, keepdims=1
21	ScatterND / ScatterElements	ScatterElements	dim ≤ 4, width 축만, updates channel 1, data는 zero 상수, reduction 제한
22–25	Squeeze, Tanh, HardSigmoid, Elu	각 Layer	dim ≤ 6
26	Reshape	Reshape	가변 shape 불가, allowzero 불가, 입출력 volume 동일, dim ≤ 6
27	Gather	Gather	입력 > 1D, data 상수 불가(indices만 상수 가능), axis보다 높은 차원 shape 1, indices 1D
28	Transpose	Transpose	dim ≤ 6
29	LayerNormalization	LayerNorm	width 축만, scale·bias는 [N] 또는 [1,N]
30	GridSample	GridSample	nearest·bilinear, zero padding, 2D grid

No	ONNX Operator	TIDL Layer	핵심 제약
31	TopK	TopK	sorted=1, K는 initializer, height 축만, 같은 값 순서는 PC·보드 간 다를 수 있음
32	DeformConv	DeformableConv	3x3s1만
33	Clip	Clip	min ≤ 0, max > 0
34	LeakyRelu	LeakyRelu	dim ≤ 6
35	Erf / Identity	Identity	단독 불가 (Erf는 GELU 조합으로 지원)
36–37	DequantizeLinear / QuantizeLinear	Dequantize / Quantize	axis=1만, ONNX QDQ 모델에서만
38	Sqrt	Sqrt	dim ≤ 6
39	ReduceMean	ReduceMean	단독 불가 → LayerNorm 조합 또는 optimizer(`convert_reducemean_to_matmul`)
40	Pow	Pow	dim ≤ 6
41	Cast	Cast	네트워크 입력·출력 terminal 노드에서만
42–46	Asin, Asinh, HardSwish, Mish, Log	각 Layer	dim ≤ 6
47	Unsqueeze	Unsqueeze	출력 차원 ≤ 6
48–52	Abs, Floor, Exp, Sin, InstanceNormalization	각 Layer	dim ≤ 6
53	SpaceToDepth	SpaceToDepth	4D, H·W가 block 배수
54–60	Acos, Atan, Sinh, Neg, Cos, Cosh, Tan	각 Layer	dim ≤ 6

TFLite Operator (25종) 요약

Conv2d/DepthwiseConv2d, AveragePool2d/Mean/MaxPool2d, Prelu/Relu, Mul/Sub/Add/Div, FullyConnected, Softmax, Concatenation(batch 축 불가), StridedSlice(batch=1), ArgMax(axis=-3), ResizeBilinear/ResizeNearestNeighbor, DepthToSpace, Logistic, Pad(8개 pad 값), Quantize/Dequantize, Squeeze, Tanh, Elu, Reshape, Transpose, LeakyRelu, BatchToSpaceNd, SpaceToBatchNd, Pack, Cast, SpaceToDepth. — LayerNorm·GELU·LSTM 매핑 없음(Transformer는 ONNX만 지원).

부록 B. 주요 Compile / Inference Option (11_00_06_00)

Option	값 (기본값)	설명 · TDA4VM 관련 사항
`tidl_tools_path`, `artifacts_folder`	필수	Tools 경로(`${TIDL_TOOLS_PATH}`), Artifact 저장·로드 경로
`tensor_bits`	8, 16, 32 (8)	32는 PC 전용
`debug_level`	0–6 (0)	1: layer cycle(보드), 3: fixed-point trace, 4: fixed+float trace
`max_num_subgraphs`	≤ 16 (16)	초과분은 ARM 실행
`accuracy_level`	0, 1, 2, 9 (1)	9는 세부 옵션 직접 지정
`advanced_options:calibration_frames` / `calibration_iterations`	(20) / (50)	최소 10 권장, 예제 스크립트는 2 / 5로 설정되어 있음
`advanced_options:output_feature_16bit_names_list`, `params_16bit_names_list`	""	수동 Mixed Precision
`advanced_options:mixed_precision_factor`	> 1 (-1)	자동 Mixed Precision
`advanced_options:quantization_scale_type`	0, 1, 3, 4 (0)	**TDA4VM은 0/1만** (3: TFLite INT8, 4: 비대칭 — 미지원)
`advanced_options:prequantized_model`	0/1 (0)	ONNX QDQ 모델 scale 사용
`advanced_options:high_resolution_optimization`	0/1 (0)	고해상도 모델 성능 최적화
`advanced_options:add_data_convert_ops`	0–3 (0)	입출력 layout·type 변환을 DSP에서 수행(실험적). TIDL-RT C API 사용 시 3 필요
`advanced_options:c7x_firmware_version`	("11_00_00_00")	패치 firmware 사용 시 명시
`advanced_options:batch_mode`	0/1 (0)	Batch stitching (3.4 B8)
`advanced_options:inference_mode`, `num_cores`, `core_number`, `core_start_idx`	(0), (1)	Multi-DSP SoC 전용 — TDA4VM은 기본값 유지
`deny_list:layer_type` / `deny_list:layer_name` / `allow_list:layer_name`	문자열	Offload 강제 제외·포함 (allow·deny 동시 사용 불가)
`object_detection:meta_arch_type`, `object_detection:meta_layers_names_list`, `model_type`	(-1), (""), ("")	검출 후처리 가속
`advanced_options:max_num_subgraph_nodes`	(864)	Subgraph 당 최대 노드 수
`advanced_options:enable_custom_layers`, `custom_layers_names_list`	(0), ("")	C7x 사용자 Layer — 별도 add-on 패키지·Firmware 빌드 필요

부록 C. 참고 문서

저장소 / 버전	파일
edgeai-tidl-tools @ 11_00_06_00	README.md · docs/supported_ops_rts_versions.md · docs/version_compatibility_table.md · docs/tidl_fsg_quantization.md · docs/tidl_fsg_multi_c7x.md · docs/tidl_fsg_od_meta_arch.md · docs/tidl_fsg_vtfr.md · docs/tidl_fsg_io_tensors_format.md · docs/tidl_osr_debug.md · docs/custom_model_evaluation.md · docs/advanced_setup.md · docs/update_target.md · examples/osrt_python/README.md · examples/osrt_python/model_configs.py · examples/osrt_python/common_utils.py · examples/osrt_python/ort/onnxrt_ep.py · examples/osrt_python/tfl/tflrt_delegate.py · examples/tidlrt_cpp/README.md · osrt-model-tools/osrt_model_tools/onnx_tools/tidl_onnx_model_optimizer/README.md · setup.sh · requirements_pc.txt
edgeai-tidl-tools @ 11_00_08_00	docs/supported_ops_rts_versions.md · docs/version_compatibility_table.md (정정 사항 비교)
edgeai-tidl-tools @ 11_01_xx · 11_02_xx (master 11_02_20_00)	docs/operators.md · docs/sdk_version_compatibility_table.md · docs/debugging.md · docs/faq.md · docs/quantization.md · docs/multi_c7x.md · docs/model_compilation.md · docs/tidlrt.md
edgeai-tensorlab @ r11.0	edgeai-modelzoo/README.md · edgeai-modelzoo/models/configs.yaml · edgeai-modelzoo/models/vision/detection/readme.md · edgeai-benchmark/README.md · edgeai-benchmark/configs/detection.py · detection_additional.py
Texas Instruments	TDA4VM 제품 정보 (ti.com/product/TDA4VM)

GitHub: https://github.com/TexasInstruments/edgeai-tidl-tools · https://github.com/TexasInstruments/edgeai-tensorlab